.UBAfiuba
FACULTAD DE INGENIERÍA

Sistema de control y diagnóstico Bluetooth para dispositivos conectados

Ing. Gustavo Campero

Carrera de Especialización en Sistemas Embebidos

Director: Ing. Ignacio Giuliani (Sensify)

Jurados:

Jurado 1 (pertenencia)
Jurado 2 (pertenencia)
Jurado 3 (pertenencia)

Ciudad de Rosario, septiembre de 2026

Resumen

En esta memoria se presenta el desarrollo de un sistema de comunicación local destinado a facilitar las tareas de configuración, diagnóstico y mantenimiento de heladeras comerciales conectadas. La solución permite consultar el estado del equipo, modificar parámetros y actualizar el firmware desde un dispositivo móvil mediante Bluetooth Low Energy. De este modo, se reduce la dependencia de la conectividad celular y se optimizan las tareas de soporte técnico en campo.

Para su implementación fueron fundamentales los conocimientos adquiridos en la carrera tales como el concepto de modularización, arquitecturas de software y testing.

Agradecimientos

Esta sección es para agradecimientos personales y es totalmente **OPCIONAL**.

Índice general

Resumen I

1. Introducción general 1
- 1.1. Sistema Sensify para heladeras comerciales . . . 1
- 1.2. Contexto y motivación . . . 2
- 1.3. Estado del arte . . . 2
- 1.4. Objetivos del trabajo . . . 3

2. Introducción específica 5
- 2.1. Ciclo de refrigeración por compresión de vapor . . . 5
- 2.2. El controlador N150 . . . 6
- 2.3. Arquitectura de conectividad del N150 . . . 7
- 2.4. Bluetooth Low Energy . . . 8
  - 2.4.1. Adecuación de BLE al N150 . . . 8
  - 2.4.2. Comunicación entre el teléfono y el controlador . . . 9

3. Diseño e implementación 11
- 3.1. Arquitectura del sistema . . . 11
  - 3.1.1. Relación con los subsistemas del N150 . . . 12
  - 3.1.2. Flujo de procesamiento . . . 13
- 3.2. Diseño detallado por módulos . . . 13
  - 3.2.1. `ble-Main` . . . 14
  - 3.2.2. `ble-Gap` y `ble-Gatt` . . . 15
  - 3.2.3. `ble-rcvMsg` y `ble-Security` . . . 15
  - 3.2.4. `ble-Data` y `ble-Params` . . . 15
  - 3.2.5. `ble-Commands` y `ble-Logs` . . . 16
- 3.3. Configuración BLE implementada . . . 16
  - 3.3.1. Selección de la pila y del controlador . . . 16
  - 3.3.2. Opciones del host NimBLE . . . 16
  - 3.3.3. Uso de memoria externa . . . 17
- 3.4. Diseño de la capa GAP . . . 18
  - 3.4.1. Identificación del equipo . . . 18
  - 3.4.2. Contenido del *advertising* . . . 18
  - 3.4.3. Modos de operación . . . 19
  - 3.4.4. *Advertising* con datos . . . 20
  - 3.4.5. Gestión de la conexión . . . 21
- 3.5. Diseño de la capa GATT . . . 22
  - 3.5.1. Organización de servicios y características . . . 22
  - 3.5.2. Diseño orientado a la extensión . . . 23
  - 3.5.3. Comandos, estados y formatos . . . 24
  - 3.5.4. Transmisión de respuestas . . . 25
- 3.6. Capa de autenticación y encriptación . . . 26

3.6.1. Motivos del diseño en la capa de aplicación 26
3.6.2. Autenticación de la sesión 26
3.6.3. Protección de los mensajes 27
3.6.4. Gestión de la sesión y de las fallas 27
3.7. Integración del sistema 28
3.7.1. Histórico y sincronización horaria 28
3.7.2. Parámetros 29
3.7.3. Pruebas de actuadores 30
3.7.4. Metadatos y diagnóstico 31
3.7.5. Comandos para la lógica de nube y conectividad remota . . 31
3.7.6. Registros 32

4. Ensayos y resultados **33**
4.1. Pruebas funcionales del hardware 33

5. Conclusiones **35**
5.1. Conclusiones generales 35
5.2. Próximos pasos 35

Bibliografía **37**

Índice de figuras

1.1. Presencia regional de Sensify 1

2.1. Ciclo de refrigeración 6
2.2. Controlador N150 6
2.3. Placa interna del N150 7
2.4. Arquitectura de conectividad del N150 8

3.1. Arquitectura general del firmware N150 e integración del módulo `sfy-BLE`. 12
3.2. *Display* virtual de una heladera en la aplicación móvil. 20
3.3. Flujo del *advertising* con datos. 21
3.4. Sincronización horaria y descarga del histórico 29
3.5. Escritura de parámetros 30
3.6. Recorrido genérico de un comando 31

Índice de tablas

1.1. Comparación de soluciones 3

2.1. Relación entre los requerimientos de la interfaz local y las características de BLE. 9

3.1. Responsabilidades de los módulos del componente BLE. 14
3.2. Opciones del host NimBLE revisadas. 17
3.3. Distribución del contenido del *advertising*. 19
3.4. Modos de operación definidos por `APP_LINK`. 19
3.5. Servicios y características GATT implementados. 22
3.6. Comandos binarios de la interfaz GATT. 24
3.7. Campos de la trama de fragmentación. 25
3.8. Integración de las funciones de la interfaz local con los subsistemas del N150. 28

Dedicado a... [OPCIONAL]

Capítulo 1

Introducción general

En este capítulo se presenta el contexto del sistema desarrollado para heladeras comerciales, la problemática que motivó el trabajo y los principales antecedentes relacionados. Finalmente, se establecen los objetivos y el alcance del desarrollo.

1.1. Sistema Sensify para heladeras comerciales

Sensify S. A. S. [1] es una empresa dedicada al análisis de datos que trabaja junto con empresas de alimentos y bebidas de América Latina para convertir heladeras comerciales en fuentes de información que permitan gestionar sus ventas de manera inteligente y sostenible. Uno de sus productos centrales es el controlador N150 [2], un dispositivo diseñado para integrarse en heladeras comerciales con el propósito de supervisar su funcionamiento y transmitir información relevante hacia plataformas remotas mediante conectividad celular (2G y 4G). Sus componentes y su arquitectura se describen en el capítulo 2.

El controlador integra en una única plataforma funciones de control, monitoreo y conectividad [2]. Esta integración permite disponer de información sobre el estado operativo de las heladeras instaladas y facilita las tareas de diagnóstico y mantenimiento. Además, la información obtenida contribuye a la gestión técnica y operativa del parque de equipos conectados. La figura 1.1 muestra la expansión regional de la solución entre 2022 y 2025.

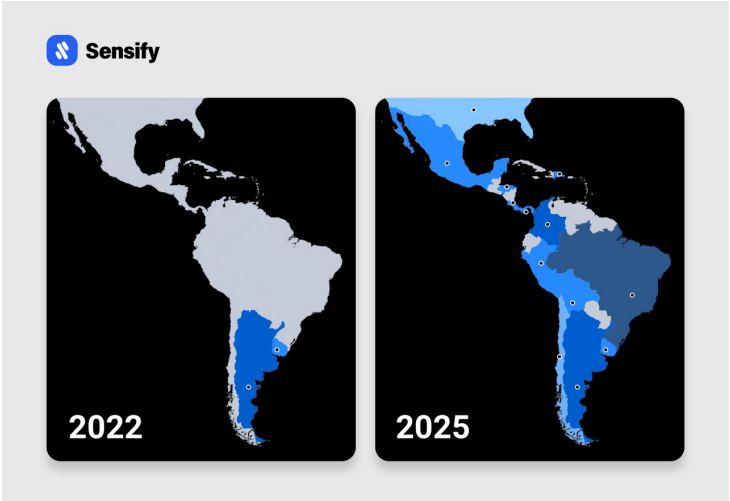


FIGURA 1.1. Presencia regional de Sensify en 2022 y 2025. Imagen cedida por Sensify S. A. S.

De acuerdo con datos internos de Sensify correspondientes a septiembre de 2026, la plataforma contaba con un parque aproximado de 70 000 heladeras conectadas, frente a las 8 000 unidades registradas en 2023. Para esa fecha, 8 fabricantes de heladeras habían integrado la tecnología de Sensify en sus equipos y la solución se encontraba presente en 10 países.

1.2. Contexto y motivación

La motivación del trabajo surgió de una limitación observada durante las tareas de configuración y soporte técnico del N150. Si bien la conectividad celular permitía atender las necesidades de telemetría y transmisión remota de datos, presentaba limitaciones para las tareas que requerían una interacción inmediata con el equipo. Entre estas tareas se encontraban la activación inicial, la modificación de parámetros, el diagnóstico técnico y la actualización de firmware.

Ante esta situación, se identificó la necesidad de disponer de un mecanismo de comunicación local que permitiera efectuar dichas operaciones de forma directa, rápida y confiable, sin depender de la cobertura ni de la estabilidad de la red celular. Este acceso resultaba especialmente relevante durante la instalación, la validación en planta y el soporte técnico en campo, ya que la disponibilidad inmediata de un canal de comunicación condicionaba la eficiencia de cada intervención.

La incorporación de un canal local también ofrecía una oportunidad de mejora operativa. Por este motivo, se desarrolló un sistema de comunicación basado en Bluetooth para interactuar con el controlador mediante un dispositivo móvil de uso común. El sistema tuvo como finalidad reducir la necesidad de herramientas específicas, simplificar las tareas de mantenimiento y ampliar las capacidades de soporte sobre los equipos instalados.

Desde el punto de vista formativo, el desarrollo permitió abordar un problema real de ingeniería mediante la integración de conceptos de sistemas embebidos, comunicaciones inalámbricas, arquitectura de software y diseño de interfaces de usuario.

1.3. Estado del arte

Las soluciones conectadas aplicadas a la refrigeración comercial adoptan distintos enfoques según la conectividad empleada y las funciones disponibles. Algunas se concentran en la adquisición de variables y el monitoreo remoto, mientras que otras incorporan funciones de configuración, diagnóstico o control. Para analizar estos enfoques se consideraron soluciones comerciales relacionadas con la supervisión de equipos refrigerados y el acceso local a sistemas embebidos.

Entre las soluciones comerciales orientadas al monitoreo se encuentran Swift Sensors [3] y Monnit [4]. Ambas ofrecen sensores inalámbricos, plataformas de supervisión remota y generación de alertas. Swift Sensors utiliza Bluetooth entre los sensores y una puerta de enlace que transmite la información hacia una plataforma remota. Monnit dispone de sensores, puertas de enlace y herramientas de gestión para distintas variables ambientales y operativas. La documentación consultada de estas plataformas se concentra en la adquisición, el registro y la visualización de datos, pero no describe funciones de configuración profunda ni de actualización del software embebido del controlador refrigerado.

Otro enfoque consiste en proporcionar acceso local mediante tecnologías de corto alcance. El controlador X35P de Coel admite, de manera opcional, la programación de parámetros mediante comunicación de campo cercano (NFC, del inglés, *Near Field Communication*) [5]. Este mecanismo simplifica la configuración en el lugar de instalación y no requiere una red de datos externa. Sin embargo, exige proximidad física y no está orientado al monitoreo remoto continuo.

Las soluciones basadas en conectividad celular evitan depender de la red local del establecimiento. AoFrio ofrece controladores conectados y la plataforma AoFrio iQ, que proporciona monitoreo, diagnóstico y control remoto de equipos de refrigeración comercial [6, 7]. Este esquema resulta apropiado para administrar flotas distribuidas, aunque su funcionamiento remoto depende de la cobertura y de la calidad de la señal celular. En ubicaciones con servicio deficiente, la comunicación puede demorarse o interrumpirse.

La tabla 1.1 resume las características relevantes de las soluciones analizadas. La comparación distingue entre monitoreo, configuración o control y disponibilidad de acceso local, debido a que estas funciones responden a necesidades operativas diferentes.

TABLA 1.1. Comparación de las soluciones comerciales analizadas.

Solución	**Conectividad**	**Funciones principales**	**Limitación relevante**
Swift Sensors	Bluetooth y puerta de enlace	Monitoreo remoto, registro y alertas	No documenta configuración profunda del controlador
Monnit	Red inalámbrica y puerta de enlace	Monitoreo remoto, registro y alertas	No documenta actualización de firmware del controlador
Coel X35P	NFC y RS485	Configuración local de parámetros y control	Requiere proximidad y no brinda monitoreo remoto continuo
AoFrio iQ	Celular	Monitoreo, diagnóstico y control remoto	Depende de la cobertura celular

En el N150, la memoria externa permite conservar la información generada durante los períodos sin conectividad y transmitirla cuando se restablece el enlace. Este mecanismo resulta adecuado para la transmisión diferida de telemetría, pero no proporciona el acceso inmediato requerido durante la instalación y el soporte técnico. Entre los antecedentes analizados no se identificó una solución que reuniera la conectividad celular existente en el N150 con un canal local destinado al diagnóstico, la configuración y la actualización de firmware. Esta diferencia fundamentó la incorporación de Bluetooth como canal complementario, sin reemplazar el enlace celular utilizado para la operación remota.

1.4. Objetivos del trabajo

El objetivo general del trabajo fue diseñar e implementar un sistema de comunicación local basado en Bluetooth para el controlador Sensify N150. El sistema

debía complementar la conectividad remota existente y permitir la ejecución de tareas de soporte técnico, configuración y mantenimiento sin depender de la disponibilidad de la red celular.

Para alcanzar el objetivo general se definieron los siguientes objetivos específicos:

- Incorporar en el firmware del ESP32 un módulo que permitiera establecer una comunicación inalámbrica local con dispositivos móviles.
- Permitir la consulta de información de estado y diagnóstico del controlador mediante el nuevo canal de comunicación.
- Permitir la modificación de parámetros operativos y la activación inicial del equipo.
- Implementar un mecanismo para actualizar localmente el firmware mediante Bluetooth.
- Integrar la nueva funcionalidad en la arquitectura existente del N150 sin interferir con los mecanismos de conectividad previamente implementados.
- Administrar la memoria dinámica y los recursos compartidos utilizados por los distintos módulos de software.
- Verificar mediante pruebas funcionales la comunicación, las operaciones de soporte implementadas y la estabilidad general del sistema.

Estos objetivos establecieron resultados verificables para la comunicación local, las funciones disponibles y la integración con el sistema existente. Los criterios y procedimientos utilizados para su evaluación se presentan en los capítulos posteriores.

Capítulo 2

Introducción específica

En este capítulo se presentan los conceptos técnicos que fundamentan el trabajo, entre ellos el ciclo de refrigeración y la función del controlador, el hardware del N150, su arquitectura de conectividad y las tecnologías de comunicación relevantes para implementar una interfaz local.

2.1. Ciclo de refrigeración por compresión de vapor

Para comprender el entorno de aplicación del controlador N150 y las tareas de soporte asociadas con su operación, resulta necesario introducir el principio de funcionamiento de los sistemas de refrigeración por compresión de vapor, tecnología ampliamente utilizada en heladeras comerciales [8].

El objetivo de estos sistemas es extraer calor del interior del gabinete refrigerado y disiparlo hacia el ambiente exterior, a fin de mantener una temperatura adecuada para la conservación de los productos almacenados. Para ello, el ciclo se basa en la circulación de un fluido refrigerante a través de un conjunto de componentes, cada uno con una función específica dentro del proceso térmico.

El primer componente del ciclo es el compresor, encargado de aspirar el refrigerante en estado gaseoso y a baja presión desde la salida del evaporador, y comprimirlo hasta alcanzar valores elevados de presión y temperatura. A continuación, el fluido ingresa al condensador, donde cede calor al ambiente y se transforma en líquido a alta presión. Luego atraviesa un dispositivo de expansión, que reduce su presión y temperatura. Finalmente, el refrigerante ingresa al evaporador, donde absorbe calor del interior de la heladera y vuelve a evaporarse, completando el ciclo. Este proceso se representa en la figura 2.1.

Desde el punto de vista funcional, el desempeño del sistema depende no solo del estado operativo de sus componentes, sino también de la supervisión adecuada de variables como la temperatura, los tiempos de funcionamiento, el estado de los sensores, las maniobras de descongelamiento y la activación de los actuadores. En aplicaciones comerciales, esta supervisión resulta fundamental para garantizar la conservación de los productos, optimizar el consumo energético y detectar fallas de manera temprana.

En el caso particular del N150, el controlador interviene en la lógica de regulación del ciclo de refrigeración y centraliza funciones de supervisión, almacenamiento de eventos, parametrización operativa y conectividad. De este modo, el dispositivo actúa como nexo entre el comportamiento físico del sistema de refrigeración y las funciones de monitoreo, diagnóstico y soporte construidas sobre él.

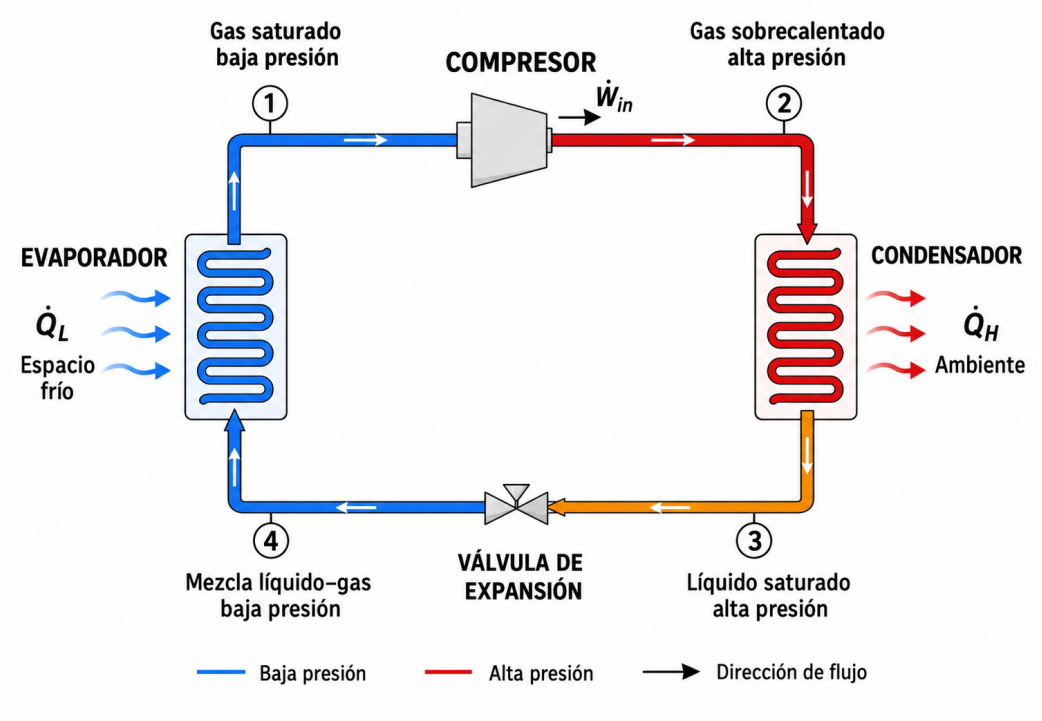


FIGURA 2.1. Esquema del ciclo de refrigeración: compresor, condensador, dispositivo de expansión y evaporador.

2.2. El controlador N150

El N150 es un producto preexistente de Sensify. Su hardware y sus subsistemas de control y conectividad celular no forman parte de este trabajo, sino que constituyen la plataforma sobre la que se integró la interfaz local. La figura 2.2 muestra el controlador. En su frente dispone de un visor, indicadores de estado y teclas para la operación local del equipo.



FIGURA 2.2. Controlador N150. Imagen cedida por Sensify S. A. S.

Desde el punto de vista arquitectónico, el controlador se basa en la interacción de distintos componentes especializados. Incorpora el PIC24FJ64GU205T [9], un microcontrolador encargado de ejecutar la lógica de control eléctrico y mecánico del

sistema de refrigeración. También integra un microcontrolador ESP32 [10], responsable de la conectividad y del procesamiento asociado a las comunicaciones. El ESP32 cuenta con 2 MB de memoria externa de tipo pseudoestática (PSRAM, del inglés, *Pseudo-Static Random Access Memory*), que amplía la memoria interna del microcontrolador. Para el acceso a redes celulares mediante comandos AT, cuenta con un módulo Quectel BG95 [11]. Además, dispone de una memoria externa para almacenar información y respaldar datos cuando las condiciones de conectividad no permiten su transmisión inmediata.

La figura 2.3 muestra la placa interna del N150 sobre la que se realizó el desarrollo. En ella se identifican el módulo ESP32 y el módem Quectel BG95.

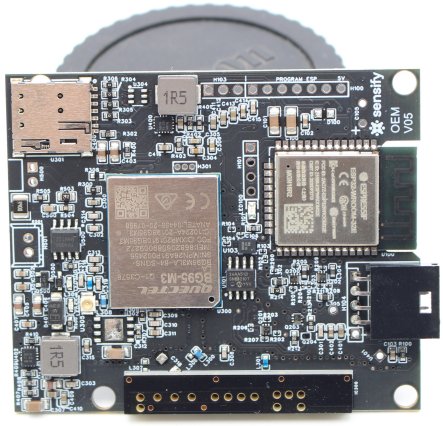


FIGURA 2.3. Placa interna del N150 con el módulo ESP32 y el módem Quectel BG95. Imagen cedida por Sensify S. A. S.

Esta estructura proporciona capacidades de telemetría, parametrización remota, actualización de firmware y geolocalización de las heladeras instaladas. Con estas funciones, el N150 opera no solo como controlador del equipo de frío, sino también como nodo inteligente dentro de una red de activos conectados.

2.3. Arquitectura de conectividad del N150

La arquitectura del N150 separa las funciones de regulación del sistema de refrigeración de las funciones de conectividad. El subsistema de control intercambia información con el ESP32, que concentra las comunicaciones y vincula el funcionamiento interno del equipo con las interfaces local y remota. Esta organización evita que las tareas de conectividad interfieran con las funciones de regulación y permite modificar los mecanismos de comunicación sin alterar la lógica principal del equipo.

La comunicación remota se realiza mediante un módem celular Quectel BG95, controlado por el ESP32 a través de una interfaz serie y comandos AT. Esta interacción permite inicializar el módulo, verificar el estado de la tarjeta SIM, registrar el equipo en la red, establecer la conexión de datos y transmitir información hacia la infraestructura de Sensify. La disponibilidad del enlace depende de la cobertura de las redes celulares admitidas por la variante del módem instalada.

La figura 2.4 presenta la relación entre los subsistemas y los canales de comunicación del N150.

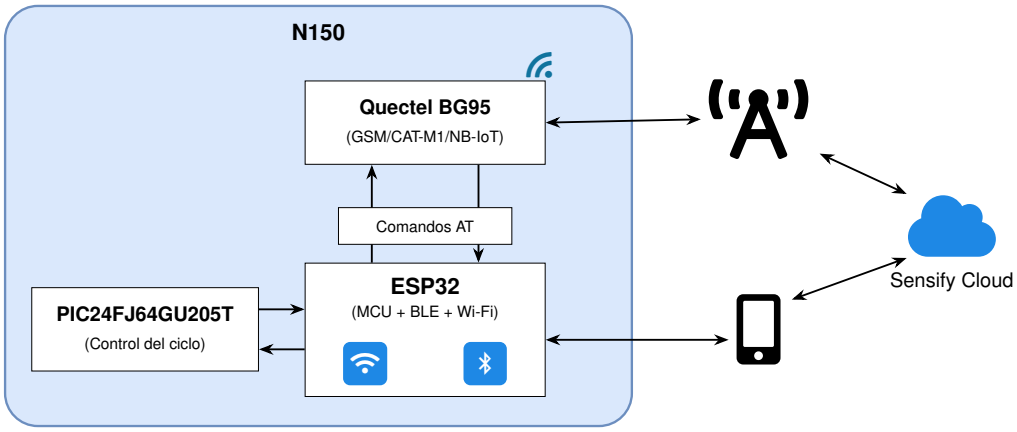


FIGURA 2.4. Arquitectura de conectividad del controlador N150.

Para el acceso local se utiliza la interfaz Bluetooth de baja energía (BLE, del inglés, *Bluetooth Low Energy*) integrada en el ESP32 y gestionada mediante ESP-IDF 5.3.2. Este canal permite que un teléfono se comunique directamente con el controlador sin utilizar el módem celular ni una red del establecimiento.

2.4. Bluetooth Low Energy

BLE es una tecnología inalámbrica de corto alcance orientada al intercambio eficiente de cantidades moderadas de datos [12]. Su especificación define, entre otros elementos, los procedimientos de descubrimiento y conexión, los roles de los dispositivos y el modelo utilizado para organizar la información intercambiada. La documentación de ESP-IDF 5.3.2 describe el soporte provisto por el ESP32 para implementar estos mecanismos [13]. Para este trabajo no resultan relevantes todas las capas del protocolo, sino aquellas que permiten establecer una comunicación directa y segura entre el N150 y un teléfono.

2.4.1. Adecuación de BLE al N150

Para el desarrollo se requería un canal local que permitiera configurar el controlador, consultar información de diagnóstico y actualizar su firmware aun cuando la heladera estuviera instalada en un sitio sin cobertura celular. También, debía posibilitar el uso de un teléfono como herramienta de soporte, sin incorporar hardware adicional al N150. BLE satisface estos requerimientos porque se encuentra integrado en el ESP32, dispone de soporte en dispositivos móviles y permite descubrir y conectar equipos cercanos sin depender de una infraestructura de red externa.

El alcance acotado de BLE también resulta coherente con las tareas de intervención local, en las que el operador se encuentra próximo a la heladera. El caudal disponible es suficiente para intercambiar parámetros, estados, registros de diagnóstico y bloques de una actualización de firmware. Además, su modelo de servicios y características, definido por el perfil genérico de atributos (GATT, del inglés, *Generic Attribute Profile*), permite organizar esas operaciones mediante una interfaz común para el controlador y la aplicación móvil.

Frente a la conectividad celular, BLE proporciona acceso cuando no existe cobertura o cuando el enlace remoto no se encuentra disponible. A diferencia de Wi-Fi, no requiere que el controlador se incorpore a una red local ni que se conozcan sus credenciales. Por otra parte, ofrece una sesión de comunicación y un modelo de datos más apropiados para las tareas previstas que una interfaz de proximidad basada en NFC. Bluetooth Classic proporciona un caudal mayor, pero ese beneficio no resulta necesario para el volumen de información de esta aplicación. Por estos motivos, se seleccionó BLE como interfaz local del N150.

La tabla 2.1 relaciona las necesidades principales de la interfaz local con las características de BLE aprovechadas en el desarrollo. Esta correspondencia permite justificar la selección a partir de los requerimientos del sistema y no solo de las propiedades generales de la tecnología.

TABLA 2.1. Relación entre los requerimientos de la interfaz local y las características de BLE.

Requerimiento	**Característica utilizada**
Operación sin cobertura celular	Enlace directo entre el teléfono y el N150, independiente de servicios externos.
Uso de un teléfono como herramienta	Compatibilidad de BLE con dispositivos móviles y mecanismos de descubrimiento de equipos próximos.
Ausencia de hardware adicional	Interfaz de radio integrada en el ESP32 del controlador.
Configuración y diagnóstico	Organización de datos y operaciones mediante servicios y características GATT.
Actualización de firmware	Transferencia bidireccional de bloques de datos durante una conexión establecida.

2.4.2. Comunicación entre el teléfono y el controlador

La comunicación comienza con la emisión periódica de paquetes de publicidad por parte del N150. El teléfono explora los dispositivos cercanos, identifica el controlador y solicita el establecimiento de una conexión, de acuerdo con el flujo previsto por la plataforma para el descubrimiento y la conexión BLE [13]. En este esquema, el N150 funciona como periférico y el teléfono como central. La publicidad permite iniciar la comunicación sin conocer previamente la dirección del dispositivo y limita la búsqueda al entorno próximo del operador.

La sesión de soporte local se divide en tres etapas. En la primera, la aplicación identifica el N150 mediante la información incluida en sus paquetes de publicidad. En la segunda, ambos dispositivos establecen la conexión y aplican los mecanismos de seguridad previstos para verificar el acceso. En la tercera, la aplicación descubre la interfaz GATT y ejecuta la operación solicitada. Esta secuencia se utiliza tanto para intervenciones breves, como la consulta de un estado, como para operaciones prolongadas, entre ellas la transferencia de una actualización de firmware.

Una vez establecida la conexión, los datos se intercambian mediante GATT. El N150 funciona como servidor GATT y expone la información y las operaciones

disponibles mediante servicios y características. El teléfono funciona como cliente GATT, descubre esa estructura y accede a ella de acuerdo con los permisos definidos.

Los roles de conexión y los roles GATT describen aspectos diferentes y complementarios. La condición de periférico establece la participación del N150 en el descubrimiento y en la conexión BLE, mientras que la condición de servidor determina que conserva y expone los atributos. De manera análoga, el teléfono actúa como central porque inicia la conexión y como cliente porque solicita operaciones sobre los atributos publicados por el controlador.

La interfaz implementada utiliza las cuatro operaciones de acceso requeridas por el sistema: lectura, escritura, notificación e indicación. Las lecturas permiten que el teléfono solicite valores al controlador y las escrituras permiten enviarle datos o comandos. Las notificaciones y las indicaciones posibilitan que el N150 informe cambios o resultados sin esperar una consulta periódica; las indicaciones, a diferencia de las notificaciones, requieren confirmación de recepción. Estas operaciones sustentan las funciones de parametrización, diagnóstico y actualización de firmware desarrolladas para la aplicación móvil.

Capítulo 3

Diseño e implementación

Este capítulo describe la arquitectura y la organización interna del módulo de comunicación BLE desarrollado para el controlador N150. Se presenta su integración con el firmware existente, se detallan los módulos que componen `sfy-BLE` y se describe la configuración BLE implementada.

3.1. Arquitectura del sistema

El desarrollo se incorpora al firmware del N150 como el componente `sfy-BLE`. Su responsabilidad es proporcionar una interfaz de comunicación local basada en BLE para las tareas de soporte del controlador. El componente recibe solicitudes de un cliente BLE externo, las adapta a las interfaces disponibles en el firmware y publica las respuestas correspondientes. El cliente utiliza el módulo como servidor GATT, pero no forma parte del sistema desarrollado en este trabajo.

El alcance del trabajo se limita a `sfy-BLE`. Los subsistemas que gestionan el funcionamiento de la heladera, la persistencia de datos, los parámetros, la conectividad remota y los registros ya se encontraban desarrollados en el firmware del N150. Por este motivo, el módulo BLE no incorpora una segunda implementación de esas capacidades ni modifica sus responsabilidades. Su aporte consiste en habilitar un acceso local que reutiliza los servicios existentes.

La arquitectura general se organiza alrededor de `sfy-BLE`, destacado como el componente desarrollado, y de los módulos preexistentes del N150. La pila NimBLE provee el acceso a la radio y a los mecanismos BLE sobre los que se apoya `sfy-BLE`. A través de esta pila, el componente anuncia el equipo, atiende la conexión del cliente y administra la interfaz GATT.

Dentro de `sfy-BLE`, `ble-Main` administra el ciclo de vida de NimBLE. `ble-Gap` configura el *advertising* y la conexión, mientras que `ble-Gatt` define los servicios y las características mediante los que se intercambian las solicitudes y respuestas con el cliente. Estos módulos constituyen la interfaz de comunicación local y coordinan el acceso a los módulos internos de procesamiento.

La figura 3.1 presenta los bloques principales y el flujo de procesamiento entre ellos. El recuadro exterior delimita el firmware del N150 y deja fuera al cliente BLE, que no forma parte del desarrollo. El recuadro de trazo grueso agrupa los módulos de `sfy-BLE`, y los bloques ubicados fuera de él corresponden a componentes preexistentes. Las líneas continuas indican intercambios directos y las punteadas, integraciones mediante callbacks.

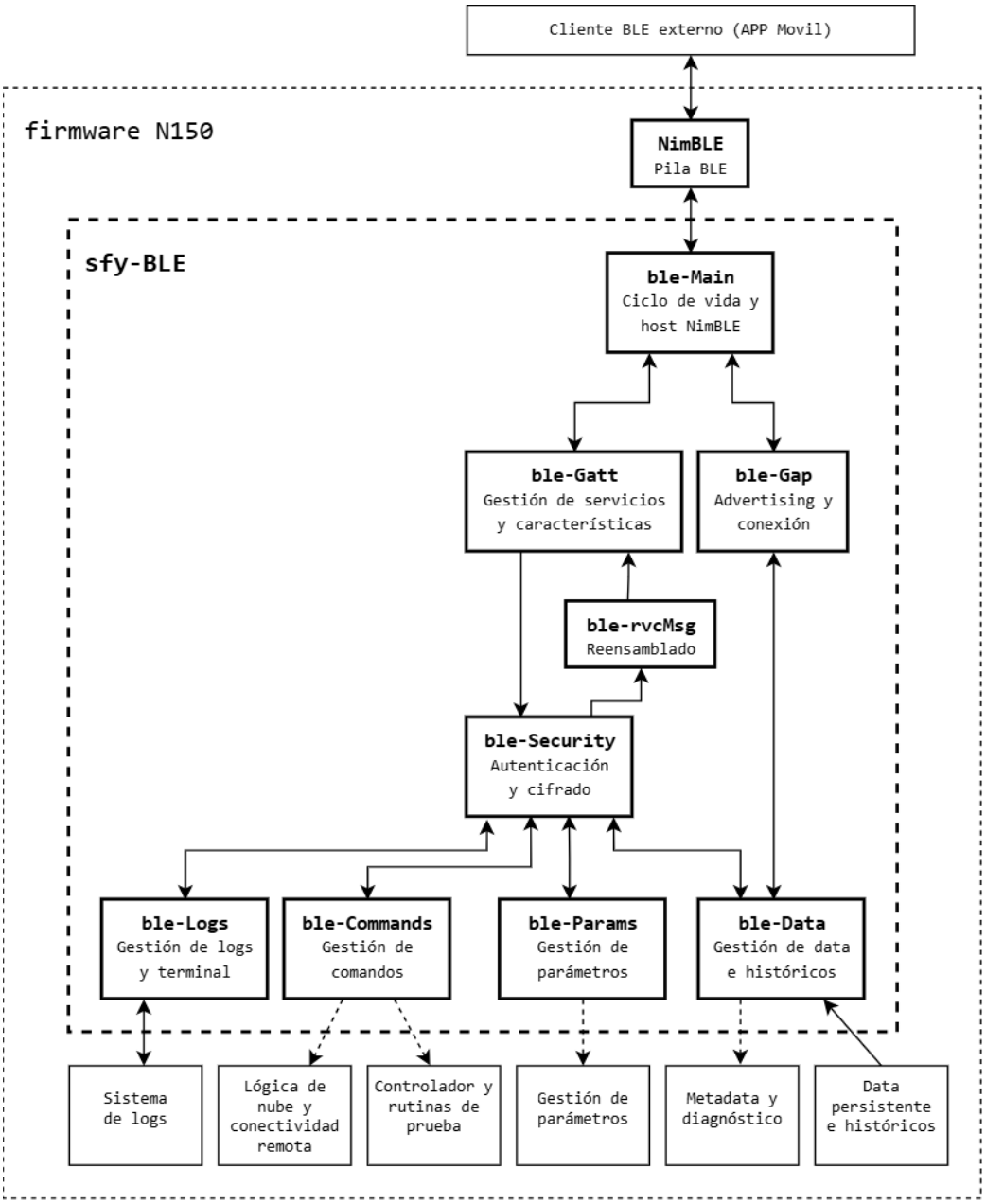


FIGURA 3.1. Arquitectura general del firmware N150 e integración del módulo `sfy-BLE`.

3.1.1. Relación con los subsistemas del N150

`sfy-BLE` se integra con el firmware mediante dos mecanismos distintos. El primero utiliza callbacks registrados durante la inicialización. Mediante estos callbacks, el módulo solicita la generación de metadatos y diagnóstico, la lectura y el procesamiento de parámetros, la ejecución de comandos locales y el encolado de comandos destinados a la lógica de nube y conectividad remota. Este mecanismo evita que `sfy-BLE` dependa directamente de las implementaciones internas de estos servicios.

La gestión de parámetros conserva, por lo tanto, las validaciones y el flujo de actualización que ya utilizaba el N150. De manera equivalente, las órdenes de control se resuelven mediante las rutinas existentes de prueba de los actuadores. Los

comandos que deben alcanzar la infraestructura remota se entregan a la lógica de nube y conectividad remota para que sigan el mismo camino de procesamiento que los comandos recibidos por los medios de conectividad habituales.

El segundo mecanismo corresponde a dependencias directas del componente. `sfy-BLE` utiliza el subsistema de *data* para obtener el último registro de telemetría y transferir el histórico almacenado. También se vincula con el sistema de registros para publicar mensajes generales y la salida de terminal durante una intervención de soporte. Estas dependencias forman parte de las operaciones internas de transferencia y depuración del módulo, por lo que no se exponen mediante callbacks del firmware principal.

La diferenciación entre ambos mecanismos responde al límite de responsabilidades definido para el desarrollo. Los callbacks se emplean cuando la operación pertenece a la lógica de negocio del N150 y debe permanecer controlada por el firmware principal. Las dependencias directas se emplean para los servicios que `sfy-BLE` necesita utilizar durante su funcionamiento interno. Esta decisión mantiene al componente desacoplado de las decisiones de control y conectividad, sin impedir el acceso a los datos y registros necesarios para la interfaz local.

3.1.2. Flujo de procesamiento

El flujo de procesamiento representado en la figura 3.1 es funcional y no corresponde a un diagrama estricto de llamadas entre archivos fuente. Las solicitudes recibidas por `ble-Gatt` que requieren fragmentación se entregan a `ble-rcvMsg`. Este módulo reensambla la secuencia recibida y, cuando completa el mensaje, lo entrega a `ble-Security`. El módulo de seguridad autentica la sesión, verifica el mensaje y recupera su contenido antes de que alcance los adaptadores funcionales.

Los módulos `ble-Commands`, `ble-Data` y `ble-Params` procesan el contenido validado según la operación solicitada. `ble-Logs` publica los registros generales y la salida de terminal utilizados durante soporte. Las operaciones de autenticación y los mensajes breves se procesan directamente entre `ble-Gatt` y `ble-Security`, sin requerir el reensamblado previo.

Una vez resuelta la solicitud, el resultado regresa a la interfaz GATT y se transmite al cliente BLE. Esta organización permite que la comunicación local sea una vía adicional de acceso al N150, sin alterar el funcionamiento interno de los módulos preexistentes. La arquitectura también facilita el mantenimiento: las modificaciones en los parámetros, el control o la lógica de nube y conectividad remota no requieren cambios en la capa BLE mientras se conserve el contrato de integración establecido.

3.2. Diseño detallado por módulos

La implementación se dividió en módulos con responsabilidades acotadas. La modularización combinó la separación de responsabilidades, la mantenibilidad y la posibilidad de realizar pruebas independientes. Esta organización evita concentrar la configuración de la pila, el procesamiento de mensajes, la seguridad y la integración con el N150 en un único componente. La tabla 3.1 resume los módulos que conforman `sfy-BLE`.

TABLA 3.1. Responsabilidades de los módulos del componente BLE.

Módulo	Responsabilidad principal
`ble-Main`	Administra el ciclo de vida de NimBLE, la tarea del host y la sincronización de los recursos compartidos.
`ble-Gap`	Configura el nombre, el *advertising*, la conexión, la desconexión y la unidad máxima de transmisión (MTU, del inglés, *Maximum Transmission Unit*).
`ble-Gatt`	Define servicios y características, procesa accesos y transmite notificaciones e indicaciones.
`ble-rcvMsg`	Reensambla mensajes fragmentados y valida secuencia, longitud y CRC32.
`ble-Security`	Implementa autenticación, claves de sesión, contadores y cifrado AES-256-GCM.
`ble-Data`	Genera metadatos, diagnóstico e histórico, y administra los datos de *advertising*.
`ble-Params`	Vincula las solicitudes de lectura y escritura con el gestor de parámetros.
`ble-Commands`	Adapta los comandos locales y los destinados a la lógica de nube y conectividad remota.
`ble-Logs`	Publica registros generales y la salida de terminal para soporte y depuración.

3.2.1. `ble-Main`

`ble-Main` administra el ciclo de vida de la pila NimBLE mediante cuatro estados: detenido, iniciando, iniciado y deteniendo. Las transiciones se protegen con un mutex recursivo compartido con `ble-Params`. El carácter recursivo permite que un cambio del modo de operación, que ya retiene el mutex, invoque el inicio o la detención de la pila sin bloquearse a sí mismo. Las solicitudes que coinciden con el estado actual se resuelven sin efectos, mientras que las que llegan durante una transición en curso se rechazan, de modo que no se reinicializan recursos que puedan estar en uso.

La inicialización recibe el serial del equipo, prepara el material criptográfico utilizado por `ble-Security` y, si el parámetro `APP_LINK` no deshabilita la interfaz, inicia la pila. El inicio construye el nombre publicado a partir del serial, inicializa NimBLE y los servicios del perfil de acceso genérico (GAP, del inglés, *Generic Access Profile*) y GATT, registra los callbacks del host y crea una tarea de FreeRTOS dedicada a ejecutar el host BLE. Esta tarea separa el procesamiento de eventos BLE del flujo principal del firmware. Si alguna de estas etapas falla, se libera lo ya inicializado y el estado vuelve a detenido. El *advertising* no comienza durante el inicio, sino cuando el host se sincroniza con el controlador. En ese momento se fija una MTU preferida de 256 bytes y se delega a `ble-Gap` la configuración del *advertising*.

La detención sigue el orden inverso. Primero se detiene el *advertising* y se cierra la conexión activa, luego se solicita la finalización del host y se espera que su tarea termine antes de liberar la pila. Si la tarea no finaliza dentro del tiempo previsto o la liberación falla, el módulo conserva el estado iniciado y no informa

una detención incompleta como exitosa. Los parámetros con los que se configura la pila se describen en la sección 3.3.

3.2.2. `ble-Gap` y `ble-Gatt`

`ble-Gap` concentra los procedimientos de descubrimiento y conexión. Construye el nombre del equipo a partir de su serial, configura los datos de *advertising* y responde a los eventos de conexión, desconexión y negociación de MTU. Al finalizar una conexión, restablece el estado necesario para que el controlador vuelva a estar disponible para una nueva intervención local. Las decisiones de diseño de esta capa se describen en la sección 3.4.

`ble-Gatt` define la interfaz de servicios y características utilizada por el cliente BLE. Sus manejadores reciben las operaciones de lectura, escritura y suscripción, y coordinan la transmisión de notificaciones e indicaciones. También constituye el punto de integración con `ble-rcvMsg`, `ble-Security` y los adaptadores funcionales. Esta división permite modificar la estructura GATT sin incorporar en sus manejadores la lógica de reensamblado, autenticación o acceso a los subsistemas del N150. Las decisiones de diseño de esta capa se describen en la sección 3.5.

3.2.3. `ble-rcvMsg` y `ble-Security`

`ble-rcvMsg` se separó de la capa GATT para reutilizar la fragmentación y sus validaciones en las características que reciben mensajes extensos. El módulo reensambla la secuencia recibida y verifica número de secuencia, longitud y CRC32 antes de entregar el mensaje completo. De este modo, los manejadores GATT permanecen enfocados en el acceso a las características y no replican el tratamiento de cada fragmento.

`ble-Security` implementa una política transversal para todas las operaciones funcionales. Administra la autenticación de sesión, las claves temporales, los contadores y el cifrado de mensajes. Mantener esta función en un módulo separado evita mezclar detalles criptográficos con los servicios GATT y permite modificar el mecanismo de protección sin alterar los adaptadores de datos, parámetros o comandos.

3.2.4. `ble-Data` y `ble-Params`

`ble-Data` concentra las operaciones de consulta. Genera metadatos del equipo, un diagnóstico de conectividad y la transferencia de información histórica almacenada. Los metadatos y el diagnóstico se obtienen mediante callbacks, mientras que el histórico y el último registro de telemetría se integran con el subsistema de *data*. El último registro también se entrega a `ble-Gap` para el modo de *advertising* con datos.

`ble-Params` atiende la consulta y el procesamiento de cambios de configuración. Ambas operaciones se delegan mediante callbacks al gestor de parámetros ya existente. Luego de una modificación, el firmware conserva su mecanismo habitual de reporte de cambios, de modo que la interfaz local no introduce un flujo de configuración paralelo.

3.2.5. `ble-Commands` y `ble-Logs`

`ble-Commands` diferencia las acciones locales de control de los comandos destinados a la infraestructura remota. Las acciones locales se delegan mediante un callback a las rutinas existentes del controlador. Los comandos remotos se incorporan a la cola de la lógica de nube y conectividad remota, lo que evita duplicar validaciones y conserva el flujo normal de procesamiento.

`ble-Logs` ofrece canales para registros generales y salida de terminal. Esta capacidad se utilizó durante el desarrollo para depurar la interfaz BLE y durante soporte para observar el comportamiento del N150 sin depender de la conectividad celular. El módulo limita la tasa de envío y conserva los mensajes de terminal en un búfer circular, con el objetivo de que la depuración no interfiera con las operaciones funcionales de la interfaz local.

3.3. Configuración BLE implementada

ESP-IDF agrupa la configuración de Bluetooth en un conjunto de opciones que se seleccionan mediante la herramienta de configuración del proyecto y se almacenan en el archivo `sdkconfig`. Estas opciones se organizan en tres niveles: la habilitación de Bluetooth y la selección de la pila del host, los parámetros del host elegido y los parámetros del controlador de radio. Cada opción tiene un valor por defecto definido por ESP-IDF. Para `sfy-BLE` se partió de esos valores y se modificaron únicamente los que no se correspondían con el uso previsto de la interfaz local. Esta sección describe las opciones disponibles, los valores por defecto de ESP-IDF 5.3.2 y los cambios realizados.

3.3.1. Selección de la pila y del controlador

ESP-IDF ofrece dos implementaciones del host Bluetooth [13]. Bluedroid, la opción por defecto, admite tanto Bluetooth clásico como BLE. NimBLE implementa únicamente BLE y requiere menos memoria y menos espacio de programa. También es posible habilitar solo el controlador, para los casos en que el host se ejecuta en otro procesador. Como la interfaz local utiliza exclusivamente BLE, se reemplazó Bluedroid por NimBLE. De este modo, el firmware no incorpora el soporte de Bluetooth clásico, que no se utiliza y ocuparía recursos compartidos con el resto de los subsistemas del N150.

El controlador del ESP32 puede operar en modo BLE, en modo Bluetooth clásico o en modo dual. Se mantuvo el valor por defecto, que habilita solo BLE y es coherente con la pila elegida. El ESP32 implementa la especificación Bluetooth 4.2 [10], por lo que ESP-IDF no habilita las funciones de Bluetooth 5, como el *advertising* extendido. En consecuencia, cada paquete de *advertising* del N150 admite como máximo 31 bytes, restricción que condiciona el diseño de la capa GAP.

3.3.2. Opciones del host NimBLE

Las opciones de NimBLE abarcan, entre otras, los roles habilitados, la cantidad de conexiones simultáneas, la memoria reservada para los mensajes de la pila, la MTU preferida y el nivel de registros. Los valores por defecto están pensados para un uso general, en el que el dispositivo puede actuar a la vez como central y como periférico y mantener varias conexiones. El N150 cumple un rol más

acotado: actúa solo como periférico y atiende a un único teléfono por vez. Por este motivo, se deshabilitaron los roles innecesarios y se redujeron las reservas de memoria, que en el ESP32 se comparten con el resto de los subsistemas del firmware. La tabla 3.2 resume las opciones revisadas, con su valor por defecto, el valor adoptado y el motivo de cada elección.

TABLA 3.2. Opciones del host NimBLE revisadas.

Opción	Valor por defecto	Valor adoptado	Motivo
Conexiones simultáneas	3	1	La interfaz local atiende a un único teléfono.
Rol periférico	Habilitado	Habilitado	Permite anunciar el equipo y aceptar conexiones.
Roles central y *broadcaster*	Habilitados	Deshabilitados	El equipo no inicia conexiones y el *advertising* conectable forma parte del rol periférico.
Bloques de memoria del primer grupo	12×256 B	12×256 B	Resultan suficientes para el tráfico de la interfaz local.
Bloques de memoria del segundo grupo	24×320 B	Ninguno	Los mensajes extensos se fragmentan en la capa de aplicación.
Búferes de datos entrantes	24×255 B	8×255 B	El tráfico entrante desde el teléfono es reducido.
MTU preferida	256 B	256 B	Coincide con la solicitada por la aplicación móvil.
Nivel de registros de la pila	Información	Advertencia	Limita los mensajes de NimBLE a advertencias y errores.
Ubicación de la memoria del host	Interna	PSRAM	Libera memoria interna (sección 3.3.3).
Nombre GAP por defecto	`nimble`	`nimble`	`sfy-BLE` lo reemplaza durante el inicio por un nombre basado en el serial.

Para cuantificar el efecto de estos ajustes se midió la memoria libre antes y después de iniciar la pila. Con la configuración utilizada al comienzo del desarrollo, que admitía tres conexiones, mantenía todos los roles y reservaba más bloques de memoria, la pila consumía 43 740 bytes. Con la configuración adoptada, el consumo se redujo a 13 980 bytes, lo que liberó 29 760 bytes para el resto del firmware.

`sfy-BLE` incluye un archivo con los valores de configuración que requiere, para que el proyecto que lo integra los incorpore a su propio `sdkconfig`. Además, la configuración se verifica durante la compilación. Si el proyecto no habilita BLE y NimBLE, o define una MTU o una cantidad de conexiones distintas de las esperadas, la compilación se interrumpe con un mensaje que indica el valor que debe corregirse. Este control evita que una diferencia de configuración se manifieste recién durante la operación del equipo.

3.3.3. Uso de memoria externa

Como se indicó en la sección 2.2, el ESP32 del N150 dispone de 2 MB de PSRAM. Hasta la incorporación de la interfaz BLE, el firmware no utilizaba esta memoria.

La pila Bluetooth de Espressif ocupa memoria interna en dos partes. El controlador reserva de forma fija 56 156 bytes de la RAM interna, que no pueden ubicarse en la memoria externa. Al iniciarse, el host asigna además sus estructuras y búferes de forma dinámica, que con la configuración adoptada ocupan los 13 980 bytes medidos. Con todas estas asignaciones en la memoria interna, el ESP32 no disponía de memoria suficiente para ejecutar la pila junto con los subsistemas preexistentes del N150.

Para resolverlo, se habilitó la PSRAM y se configuró NimBLE para asignar su memoria en ella. ESP-IDF ofrece cuatro estrategias de asignación para el host: memoria interna, que es la opción por defecto, memoria externa, la estrategia general del sistema y la RAM de instrucciones interna. Se eligió la memoria externa, de modo que la memoria interna queda disponible para el controlador y para el resto del firmware.

3.4. Diseño de la capa GAP

La capa GAP determina cómo el N150 se da a conocer a los teléfonos cercanos y cómo se establece la conexión. Su diseño partió de dos condiciones definidas en la sección 3.3: el equipo actúa solo como periférico y cada paquete de *advertising* admite como máximo 31 bytes. Dentro de ese marco, se buscó que el equipo pudiera encontrarse en cualquier momento sin intervención sobre el controlador, que su comportamiento pudiera adaptarse a cada cliente y que parte de su información pudiera consultarse sin establecer una conexión.

3.4.1. Identificación del equipo

El equipo se identifica mediante su dirección BLE y su nombre. Como dirección se utiliza la dirección pública que el ESP32 trae asignada de fábrica, por lo que un mismo controlador conserva siempre la misma identidad.

El nombre publicado se forma con el prefijo `sfy` seguido del serial del equipo. El prefijo permite que la aplicación filtre los controladores de Sensify entre todos los dispositivos BLE presentes en el lugar, y el serial permite que el técnico reconozca el equipo sobre el que debe intervenir sin necesidad de conectarse. El nombre se asigna durante el inicio de la pila y reemplaza el nombre genérico configurado en ESP-IDF.

3.4.2. Contenido del *advertising*

El N150 publica paquetes de *advertising* conectables y no dirigidos, con descubrimiento general e indicación de que no admite Bluetooth clásico. El *advertising* no tiene tiempo límite. Se mantiene mientras no haya un teléfono conectado y se reanuda cuando la conexión finaliza, cuando un intento de conexión falla o cuando la pila informa su interrupción. De este modo, el equipo puede descubrirse en cualquier momento sin que el técnico deba realizar una acción previa sobre el controlador.

El límite de 31 bytes por paquete fue una de las principales dificultades del diseño de esta capa. Como el ESP32 implementa Bluetooth 4.2, no dispone del *advertising* extendido de Bluetooth 5, que admite paquetes de mayor tamaño. En el paquete principal, los indicadores de descubrimiento ocupan 3 bytes y el campo con el

registro de telemetría, 26 bytes. Quedan entonces solo 2 bytes libres, insuficientes para incluir también el nombre del equipo.

Para resolverlo, se utilizó el *scan response*, un segundo paquete de hasta 31 bytes que el periférico entrega cuando el teléfono lo solicita mediante un *scan request* durante el escaneo. El contenido se repartió entre ambos paquetes, como resume la tabla 3.3. El paquete principal transporta los indicadores de descubrimiento y, cuando el modo de operación lo requiere, los datos de telemetría. El *scan response* transporta solo el nombre completo, de modo que su longitud no reduce el espacio disponible para los datos. El teléfono recibe ambos paquetes y los asocia al mismo equipo mediante su dirección.

TABLA 3.3. Distribución del contenido del *advertising*.

Paquete	Campo	Tamaño	Presencia
Advertising	Indicadores de descubrimiento	3 B	Siempre.
	Datos específicos del fabricante	26 B	Solo en el modo con datos.
Scan response	Nombre completo del equipo	Variable	Según el modo.

3.4.3. Modos de operación

Las instalaciones del N150 no tienen todas las mismas necesidades. Por este motivo, el comportamiento de la capa GAP se definió como configurable mediante el parámetro `APP_LINK`, administrado por el gestor de parámetros del equipo. Al ser un parámetro más del N150, puede ajustarse para cada cliente por los mismos medios que el resto de la configuración. La tabla 3.4 resume los cuatro modos disponibles.

TABLA 3.4. Modos de operación definidos por `APP_LINK`.

Modo	Datos en el *advertising*	Nombre en el *scan response*	Intervalo de *advertising*
Deshabilitado	Pila BLE detenida.		
Advertising con datos y conexión	Último registro de telemetría	Sí	500–510 ms
Advertising y conexión (predeterminado)	No	Sí	500–510 ms
Solo conexión	No	No	2000–3000 ms

El modo predeterminado corresponde al uso más general: el equipo se anuncia con su nombre cada medio segundo, lo que permite que la aplicación lo encuentre rápidamente y que el técnico se conecte. El modo de solo conexión mantiene el equipo conectable, pero no publica su nombre y espacia los anuncios entre 2 y 3 s. Así se reduce su visibilidad para quienes exploran los dispositivos cercanos y disminuye la actividad de la radio. El modo deshabilitado detiene la pila para las instalaciones que no requieren la interfaz local.

Un cambio de modo se aplica sin reiniciar el equipo. Si se deshabilita la interfaz, se detiene la pila; si se habilita, se inicia; y si se pasa de un modo activo a otro, solo se actualiza el contenido del *advertising*, sin interrumpir la publicación.

3.4.4. *Advertising* con datos

La conexión con el equipo requiere que el técnico la inicie desde la aplicación y que se complete la autenticación antes de acceder a la información. Este proceso resulta adecuado para configurar o diagnosticar un controlador en particular, pero es lento cuando solo se necesita una visión rápida del estado de varios equipos. El modo con datos se diseñó para ese caso. Al incluir la última medición en el *advertising*, la aplicación puede mostrar el estado de todas las heladeras cercanas con solo explorar el entorno, a modo de *display* virtual, sin que el técnico deba conectarse, autenticarse ni interactuar con cada equipo. La figura 3.2 muestra el *display* virtual de una heladera en la aplicación móvil. En él, las temperaturas, el estado de la puerta y de las salidas, la tensión medida, las alarmas, el tipo de red y la intensidad de señal provienen del *advertising*; los valores restantes no forman parte de él.

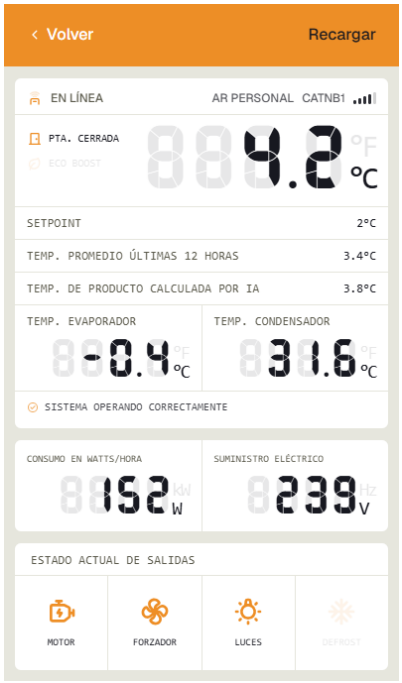


FIGURA 3.2. *Display* virtual de una heladera en la aplicación móvil.

Los datos se transmiten en el campo de datos específicos del fabricante. Su contenido es el último registro de telemetría, que `ble-Data` obtiene del subsistema de *data* y codifica con el mismo formato binario que el equipo utiliza para enviar la telemetría a la plataforma de Sensify. El registro incluye el tipo de red y la intensidad de señal, las temperaturas interna, del evaporador y del condensador, la tensión de alimentación, la marca de tiempo, indicadores del estado de la puerta, de las salidas y de las alarmas, y una suma de verificación. Codificado, ocupa 23 bytes, por lo que entra completo en el campo de 24 bytes disponible luego de los indicadores de descubrimiento. La reutilización de este formato evitó definir una codificación adicional para la interfaz local.

El contenido se actualiza cada vez que se inicia el *advertising* y cada vez que se modifica sin detenerlo. En ambos casos, `ble-Gap` solicita el registro más reciente a `ble-Data` y reemplaza el campo solo si la información cambió.

La figura 3.3 resume el flujo completo del modo con datos, desde la obtención del registro hasta su visualización en la aplicación móvil.

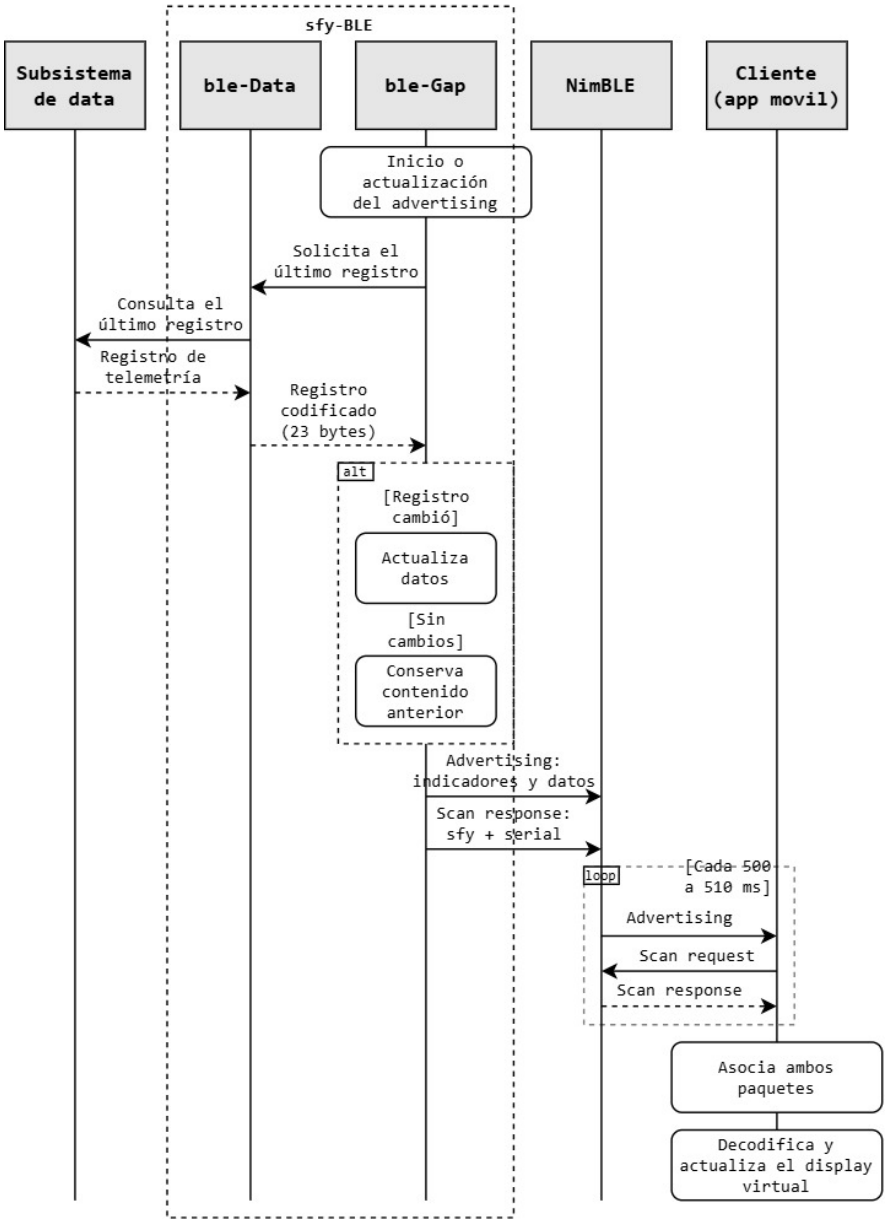


FIGURA 3.3. Flujo del *advertising* con datos.

3.4.5. Gestión de la conexión

El equipo admite una única conexión, de modo que un solo teléfono opera el N150 por vez. Al establecerse la conexión, el *advertising* se detiene, por lo que el controlador deja de aparecer para otros teléfonos hasta que la sesión finaliza.

Luego de conectarse, el N150 solicita conservar el intervalo de conexión y el tiempo de supervisión propuestos por el teléfono, con una latencia de periférico de

tres eventos. La latencia permite que el equipo omita hasta tres eventos de conexión consecutivos cuando no tiene datos para transmitir. De este modo, se estabiliza la conexión y se reduce la actividad de la radio sin modificar los tiempos negociados por el teléfono. La MTU preferida es de 256 bytes, y el valor efectivo surge de la negociación que el teléfono realiza al conectarse.

Al finalizar la conexión, `ble-Gap` descarta la sesión segura y cualquier mensaje recibido de forma parcial, y reanuda el *advertising* salvo que la interfaz haya sido deshabilitada. Esta limpieza garantiza que cada sesión comience desde un estado conocido y que la información de una sesión anterior no se reutilice en la siguiente.

3.5. Diseño de la capa GATT

La capa GATT define la interfaz mediante la cual la aplicación móvil accede a las funciones del N150 una vez establecida la conexión. Su diseño persiguió tres objetivos: organizar las operaciones por área funcional, transferir respuestas que superan el tamaño de un paquete sin perder información y facilitar la incorporación de nuevas funciones sin modificar la estructura existente.

3.5.1. Organización de servicios y características

La interfaz se organiza en cuatro servicios primarios: control, datos, parámetros y depuración. Cada servicio agrupa las características de un área funcional, como resume la tabla 3.5.

TABLA 3.5. Servicios y características GATT implementados.

Servicio	Característica	Propiedad	Uso
Control	`AUTH-SYNC`	Lectura y escritura	Establecimiento de la sesión segura.
	`COMMAND-GET`	Escritura	Comandos de control local.
	`COMMANDS-RESPONSE`	Notificación	Estado de los comandos de control.
	`CLOUDCOMMANDS-GET`	Escritura	Comandos para la lógica de nube y conectividad remota.
	`LOG-CONTROL`	Notificación	Eventos y estado de procesos.
Datos	`TIME-SYNC`	Escritura	Sincronización de fecha y hora.
	`COMMAND-DATA`	Escritura	Solicitudes de metadatos, diagnóstico, histórico y parámetros.
	`METADATA-READ`	Indicación	Metadatos del equipo.
	`HISTORY-SEND`	Indicación	Registros históricos.
	`DIAGNOSIS-READ`	Indicación	Diagnóstico de conectividad.
	`READ-STATUS`	Notificación	Estado de las lecturas en curso.
Parámetros	`PARAMETROS-GET`	Escritura	Envío de cambios de configuración.
	`PARAMETROS-RESPONSE`	Indicación	Respuesta del gestor de parámetros.
Depuración	`LOG-SEND`	Notificación	Registros generales.
	`LOG-TERMINAL`	Notificación	Salida de terminal.

Salvo en el establecimiento de la sesión, la interfaz no utiliza lecturas GATT directas. Cada operación combina una característica de escritura, por la que la aplicación envía la solicitud, con otra de indicación o notificación, por la que el equipo entrega la respuesta. Esta separación responde a dos motivos. En primer lugar,

muchas respuestas, como los metadatos, el diagnóstico o la configuración, superan el tamaño de un paquete y se generan de forma asincrónica, por lo que el equipo las envía cuando están listas y fragmentadas. En segundo lugar, las indicaciones requieren que la aplicación confirme cada recepción, lo que garantiza que un fragmento llegó antes de enviar el siguiente.

3.5.2. Diseño orientado a la extensión

La estructura GATT se diseñó para que incorporar un servicio o una característica no requiera modificar la lógica existente. Con este fin, se combinaron cuatro decisiones.

La primera es la definición declarativa de la interfaz. Los servicios y las características se describen en una única tabla, en la que cada característica es una entrada con su identificador, sus propiedades y la función que atiende sus accesos. NimBLE recorre esa tabla durante el inicio y registra la estructura completa, por lo que agregar una característica consiste en agregar una entrada.

El código 3.1 muestra un fragmento de la tabla con el servicio de datos y dos de sus características. Cada característica se define con su identificador, la función que atiende sus accesos, sus propiedades y la variable en la que NimBLE guarda su *handle*.

CÓDIGO 3.1. Fragmento de la tabla de servicios GATT.

```
static const struct ble_gatt_svc_def gatt_services[] = {
    {   /* Servicio de datos */
        .type = BLE_GATT_SVC_TYPE_PRIMARY,
        .uuid = &data_svc_uuid.u,
        .characteristics = (struct ble_gatt_chr_def[]) {
            {   /* COMMAND-DATA: solicitudes de la aplicación */
                .uuid       = &commandData_chr_uuid.u,
                .access_cb  = commandData_access_cb,
                .flags      = BLE_GATT_CHR_F_WRITE,
                .val_handle = &commandData_handle,
            },
            {   /* METADATA-READ: respuesta por indicación */
                .uuid       = &metadataRead_chr_uuid.u,
                .access_cb  = metadataRead_access_cb,
                .flags      = BLE_GATT_CHR_F_INDICATE,
                .val_handle = &g_metadataRead.attr_handle,
            },
            { 0 },  /* fin de las características */
        }
    },
    /* ... servicios de control, parámetros y depuración ... */
    { 0 },  /* fin de los servicios */
};
```

La segunda es un esquema de identificadores con base común. Los servicios y las características se identifican mediante identificadores únicos universales (UUID, del inglés, *Universally Unique Identifier*) de 128 bits que comparten una base propia de Sensify y difieren solo en el primer byte. Dentro de ese byte, la parte alta identifica el servicio y la parte baja, la característica: el servicio de control utiliza el valor `0xF0` y sus características, los valores `0xF1` a `0xF5`. Un servicio nuevo

toma el siguiente prefijo libre y una característica nueva, el siguiente valor dentro de su servicio.

La tercera es la separación entre la definición de una característica y su mecanismo de transmisión. Las indicaciones y las notificaciones se gestionan mediante dos mecanismos genéricos, descritos en la sección 3.5.4, que resuelven la suscripción, la fragmentación, los reintentos y el control de tasa. Cada característica que transmite datos solo declara un contexto con la función que entrega el próximo mensaje. En la tabla de servicios, el *handle* de esa característica se guarda dentro de su contexto, como ocurre con `g_metadataRead` en el código 3.1, lo que la vincula con el mecanismo correspondiente. La suscripción también se atiende de forma genérica: el manejador identifica la característica por su *handle* y actualiza su contexto, sin lógica específica para cada caso.

La cuarta es el uso de comandos binarios sobre características compartidas, descritos en la sección 3.5.3. Una operación nueva sobre un servicio existente puede incorporarse como un nuevo código de comando, sin crear una característica adicional.

3.5.3. Comandos, estados y formatos

Las solicitudes breves se envían como comandos binarios: un código de 2 bytes seguido de un argumento opcional de 1 byte. Los códigos se agruparon por rangos según el tipo de operación, como resume la tabla 3.6. `COMMAND-DATA` recibe las consultas y `COMMAND-GET`, las acciones de control.

TABLA 3.6. Comandos binarios de la interfaz GATT.

Código	Comando	Operación
`0x0001`	`CMD_METADATA_READ`	Lectura de metadatos.
`0x0002`	`CMD_CONNECTIVITY_READ`	Lectura del diagnóstico de conectividad.
`0x1000`	`CMD_HISTORY_GET`	Inicio de la descarga del histórico.
`0x1001`	`CMD_HISTORY_CONFIRM`	Confirmación de la descarga.
`0x1002`	`CMD_HISTORY_CANCEL`	Cancelación de la descarga.
`0x1003`	`CMD_PARAMS_READ`	Lectura de la configuración actual.
`0x3010`–`0x3015`	`CMD_CTRL_TEST_*`	Pruebas de compresor, resistencia, ventilador, luz y rutina completa.

Las operaciones asincrónicas informan su avance mediante un mensaje de estado uniforme de 3 bytes, formado por el código del comando y un valor de estado: correcto, en curso o un código de error. Las lecturas lo publican en `READ-STATUS` y las acciones de control, en `COMMANDS-RESPONSE`. De este modo, la aplicación conoce en todo momento si una operación fue aceptada, si sigue en curso o si falló, con el mismo formato para todas las operaciones. Si llega una solicitud mientras la misma transferencia está en curso, se rechaza con un estado de ocupado, en lugar de interrumpir la transferencia anterior.

Los formatos se eligieron según el tamaño y la estructura de cada mensaje. Los comandos y los estados utilizan el formato binario descrito, por su brevedad. Los

parámetros, los metadatos, el diagnóstico y los comandos para la lógica de nube y conectividad remota utilizan JSON, que se fragmenta cuando supera el tamaño de un paquete. Se mantuvo este formato porque es el que ya utilizaban los canales de conectividad remota existentes, lo que permitió reutilizar el procesamiento del firmware sin conversiones adicionales. Los registros históricos conservan el formato binario de la telemetría que el equipo envía a la plataforma. Por último, los canales de depuración no requieren comandos: la transmisión de registros comienza cuando la aplicación se suscribe a la característica y se detiene cuando cancela la suscripción.

3.5.4. Transmisión de respuestas

Las respuestas que requieren confirmación se transmiten mediante indicaciones. El mecanismo de indicaciones divide el mensaje en fragmentos y envía cada uno solo después de recibir la confirmación del anterior. Si la entrega de un fragmento falla, se reintenta hasta cinco veces antes de abortar la transferencia e informar el error por el canal de estado. Cuando la característica dispone de una función que entrega el próximo mensaje, el mecanismo la invoca al completar cada envío, lo que permite transmitir secuencias de mensajes sin cargarlas completas en memoria.

Los mensajes que superan el tamaño de un paquete se dividen en fragmentos. Cada fragmento comienza con un encabezado de texto, seguido de la porción del mensaje que transporta:

```
SEQ=<seq>;N=<total>;LEN=<len>;CRC=<crc>;F=<flag>;<payload>
```

La tabla 3.7 describe cada campo. El formato es el mismo en ambos sentidos: `ble-rcvMsg` lo valida al recibir y `ble-Gatt` lo genera al transmitir. En el sentido del equipo hacia el teléfono, el contenido de cada fragmento se limita a 128 bytes.

TABLA 3.7. Campos de la trama de fragmentación.

Campo	Contenido
`SEQ`	Número de secuencia del fragmento. Comienza en 0 y aumenta de a uno.
`N`	Cantidad total de fragmentos del mensaje.
`LEN`	Longitud total del mensaje completo, en bytes.
`CRC`	CRC32 del mensaje completo, sin los encabezados, expresado como ocho dígitos hexadecimales.
`F`	Posición del fragmento: `S` para el primero, `-` para los intermedios y `E` para el último. Un mensaje de un solo fragmento comienza directamente con `E`.
`payload`	Porción del mensaje transportada por el fragmento.

Todos los fragmentos de un mismo mensaje repiten los valores de `N`, `LEN` y `CRC`. Al recibir, `ble-rcvMsg` toma esos valores del primer fragmento y rechaza la secuencia si un fragmento llega fuera de orden o si el contenido acumulado supera la longitud declarada. El mensaje se entrega solo cuando la longitud y el CRC32 del mensaje reensamblado coinciden con los declarados.

Los estados breves y los registros se transmiten mediante notificaciones, que no requieren confirmación. Para que los registros no saturen el enlace, el mecanismo

de notificaciones limita la cantidad de mensajes por intervalo: el canal de registros generales envía como máximo 10 mensajes cada 3 s y el canal de terminal, 20 mensajes cada 300 ms.

Todas las transmisiones se encolan como eventos de NimBLE y se ejecutan en la tarea del host. Así, aunque una respuesta se origine en otra tarea del firmware, el acceso a la pila se realiza siempre desde la misma tarea, lo que evita accesos concurrentes.

3.6. Capa de autenticación y encriptación

La interfaz local permite modificar la configuración del N150, accionar sus actuadores y extraer sus datos, por lo que el acceso no puede depender solo de la proximidad física. Para restringirlo, se implementó un esquema propio de autenticación y cifrado en la capa de aplicación, en lugar de utilizar los mecanismos de emparejamiento y vinculación (*bonding*) que ofrece BLE a través de NimBLE. Esta sección describe los motivos de esa decisión y el funcionamiento general del esquema. Por tratarse de información sensible, se omiten los detalles de formato y de derivación de claves.

3.6.1. Motivos del diseño en la capa de aplicación

La decisión de no utilizar el emparejamiento de BLE respondió a cuatro motivos:

- **Experiencia del técnico.** El emparejamiento muestra diálogos del sistema operativo y requiere pasos adicionales. Con un esquema propio, la aplicación se conecta y se autentica sin intervención del técnico.
- **Muchos teléfonos por equipo.** Un mismo N150 es atendido por distintos técnicos con distintos teléfonos a lo largo de su vida útil. La vinculación de BLE almacena una cantidad limitada de vínculos, que habría que administrar en cada equipo.
- **Autorización de la aplicación.** El emparejamiento establece un enlace cifrado con cualquier teléfono que complete el procedimiento, pero no garantiza que quien se conecta sea la aplicación de Sensify. El esquema propio, basado en una clave secreta, restringe el acceso a la aplicación oficial.
- **Control del protocolo.** Al implementar la seguridad en la capa de aplicación, el comportamiento es el mismo en Android e iOS y no depende de la forma en que cada sistema operativo maneja el emparejamiento.

La implementación utiliza la biblioteca criptográfica mbedTLS, incluida en ESP-IDF, que aprovecha la aceleración por *hardware* del ESP32 para las operaciones de cifrado y *hash*, por lo que no fue necesario incorporar bibliotecas adicionales.

3.6.2. Autenticación de la sesión

La autenticación se basa en una clave secreta que solo conocen el equipo y la aplicación oficial, y se realiza al comienzo de cada conexión mediante un intercambio de desafío y respuesta sobre la característica `AUTH-SYNC`. Al establecerse la conexión, el equipo genera un valor aleatorio de un solo uso (*nonce*) que la aplicación obtiene leyendo esa característica. La aplicación genera su propio *nonce* y responde con ambos valores y un código de autenticación de mensajes basado en *hash*

(HMAC, del inglés, *Hash-based Message Authentication Code*), calculado con la clave secreta y la función SHA-256. El equipo verifica que la respuesta corresponda al desafío de la sesión en curso y que el código sea válido. Solo una aplicación que conoce la clave puede generar una respuesta correcta, y como el desafío cambia en cada conexión, una respuesta capturada no puede reutilizarse en otra.

Si la verificación es exitosa, ambos extremos derivan una clave de sesión a partir de la clave secreta y de los dos *nonces*. De este modo, cada conexión utiliza una clave distinta y la clave secreta no se emplea directamente para cifrar los mensajes.

Antes de completar la autenticación, el equipo solo acepta el intercambio sobre `AUTH-SYNC`. Cualquier escritura en otra característica provoca la desconexión, y las suscripciones a notificaciones o indicaciones se ignoran sin cerrar la conexión. Este último criterio se adoptó porque algunos clientes BLE, como ciertas aplicaciones de prueba en iOS, se suscriben automáticamente al conectarse; la aplicación vuelve a suscribirse una vez autenticada.

3.6.3. Protección de los mensajes

Una vez autenticada la sesión, todos los mensajes de la interfaz se transmiten dentro de una trama segura, en ambos sentidos. La trama cifra el contenido con AES-256-GCM, un algoritmo de cifrado autenticado que garantiza en una sola operación la confidencialidad y la integridad del mensaje. El encabezado de la trama no se cifra, pero queda protegido por la misma verificación, de modo que cualquier alteración se detecta.

Cada trama incluye un tipo de mensaje y un contador. El tipo identifica la operación, y cada característica acepta únicamente el tipo que le corresponde, lo que impide redirigir un mensaje válido a otra operación. El contador es independiente para cada sentido y debe crecer en cada mensaje: el equipo rechaza toda trama cuyo contador no supere al último recibido, lo que impide reinyectar mensajes capturados durante la sesión.

La protección se incorporó como una capa común sobre los formatos existentes. Los mensajes conservan su contenido binario o JSON, y la trama segura se fragmenta, cuando es necesario, con el mecanismo descrito en la sección 3.5.4. En consecuencia, el CRC32 de la fragmentación cumple solo una función de control de transporte, mientras que la integridad y la autenticidad quedan a cargo del cifrado autenticado. Contar con un único esquema para todas las características simplificó la implementación y las pruebas, a costa de un pequeño aumento en el tamaño de los mensajes breves.

3.6.4. Gestión de la sesión y de las fallas

La sesión segura dura lo mismo que la conexión. Al desconectarse el teléfono, `ble-Gap` descarta la clave de sesión, los contadores y cualquier mensaje parcialmente recibido, por lo que la conexión siguiente debe autenticarse nuevamente desde el inicio.

Ante cualquier falla de seguridad, como un código de autenticación inválido, un tipo de mensaje incorrecto, una trama alterada o un contador repetido, el equipo

cierra la conexión de inmediato. Todas las fallas utilizan el mismo motivo estándar de desconexión, y la causa específica se guarda solo en los registros internos del firmware.

3.7. Integración del sistema

La sección 3.1 presentó los dos mecanismos mediante los cuales `sfy-BLE` se vincula con el firmware del N150: los callbacks registrados durante la inicialización y las dependencias directas. Esta sección describe cómo cada función de la interfaz local se integra con el subsistema correspondiente. En todos los casos se siguió el mismo criterio: la interfaz BLE reutiliza los servicios existentes y no implementa una segunda versión de la lógica del N150. La tabla 3.8 resume esta integración.

TABLA 3.8. Integración de las funciones de la interfaz local con los subsistemas del N150.

Función	Subsistema del N150	Mecanismo	Características
Histórico y hora	Subsistema de *data* y servicio de hora	Dependencia directa	`HISTORY-SEND` `TIME-SYNC`
Parámetros	Gestor de parámetros	Callback	`PARAMETROS-GET` `PARAMETROS-RESPONSE`
Pruebas de actuadores	Rutinas de control	Callback	`COMMAND-GET`
Metadatos y diagnóstico	Lógica de nube y estado de conectividad	Callback	`METADATA-READ` `DIAGNOSIS-READ`
Comandos remotos	Lógica de nube y conectividad remota	Callback	`CLOUDCOMMANDS-GET`

Registros	Sistema de registros y terminal	Dependencia directa	`LOG-SEND`
			`LOG-TERMINAL`
			`LOG-CONTROL`

3.7.1. Histórico y sincronización horaria

Cuando el N150 no tiene cobertura celular, los registros de telemetría se acumulan en el almacenamiento a la espera de ser enviados a la plataforma. La descarga del histórico por BLE permite extraer esos registros pendientes con el teléfono, que pasa a ser el canal que los lleva a la plataforma.

Cada registro incluye la marca de tiempo del momento en que se generó. Si el equipo no dispone de la hora correcta, por ejemplo porque nunca obtuvo conectividad, los registros almacenados carecen de una referencia temporal válida y no pueden aprovecharse. Por este motivo, se incorporó la sincronización horaria desde el teléfono. La aplicación escribe en `TIME-SYNC` la fecha y la hora actuales en formato JSON, y `sfy-BLE` las aplica mediante el servicio de hora existente del firmware, de modo que los registros posteriores se generan con una marca de tiempo correcta.

Como los registros descargados dejan de estar pendientes para el equipo, la transferencia se diseñó como una transacción, de modo que ningún registro se pierda si la descarga se interrumpe. Al recibir la solicitud, el equipo consulta al subsistema de *data* la cantidad de registros pendientes, la envía a la aplicación y luego transmite un registro por indicación, con el mismo formato binario que utiliza para la plataforma. Cada registro enviado se marca como pendiente de confirmación, pero no se elimina del almacenamiento. Al finalizar, la aplicación confirma

la descarga y recién entonces los registros se eliminan. Si la aplicación cancela la descarga, o si transcurren dos minutos sin confirmación, los registros marcados vuelven a quedar pendientes y el equipo los enviará a la plataforma cuando recupere la conectividad. La figura 3.4 muestra la sincronización horaria y la descarga completa del histórico.

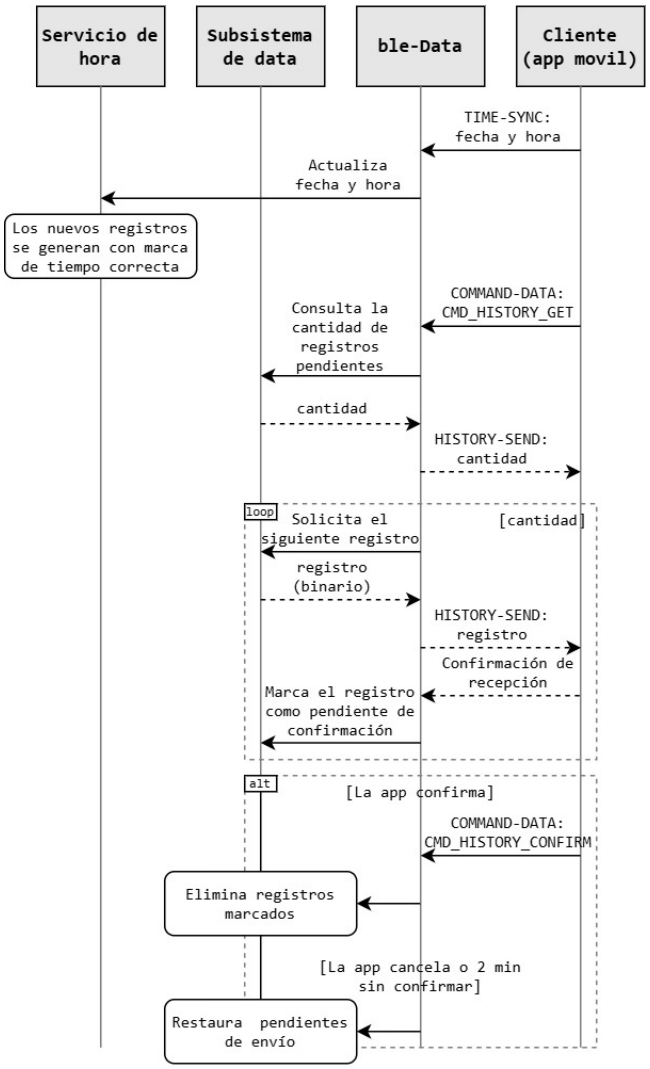


FIGURA 3.4. Sincronización horaria y descarga transaccional del histórico. La aplicación primero actualiza la hora del equipo y luego descarga los registros pendientes, que se eliminan solo si confirma la descarga y se restauran si la cancela o no responde en dos minutos.

3.7.2. Parámetros

La configuración del N150 se administra mediante un gestor de parámetros preexistente, que ya atendía los cambios recibidos desde la plataforma. La interfaz BLE se integró con él mediante dos callbacks. El de lectura obtiene la configuración actual en el mismo formato JSON que el gestor genera para la plataforma, y la aplicación la solicita con el comando de lectura de parámetros. El de procesamiento recibe los cambios que la aplicación envía por `PARAMETROS-GET`, una vez reensamblados por `ble-rcvMsg`. El gestor valida y aplica cada cambio con

las mismas reglas que utiliza para la plataforma, y su respuesta se devuelve a la aplicación por `PARAMETROS-RESPONSE`.

Luego de procesar un cambio recibido por BLE, el gestor deja pendiente un reporte forzado de la configuración, que la lógica de nube envía a la plataforma. Así, la plataforma conserva una copia actualizada de los parámetros, aunque el cambio no haya llegado por los canales remotos. La figura 3.5 muestra el recorrido completo de una escritura de parámetros. El propio componente aprovecha este mecanismo: `APP_LINK` se registra en el gestor como un parámetro más, por lo que el modo de operación de la interfaz puede modificarse por los mismos medios que el resto de la configuración.

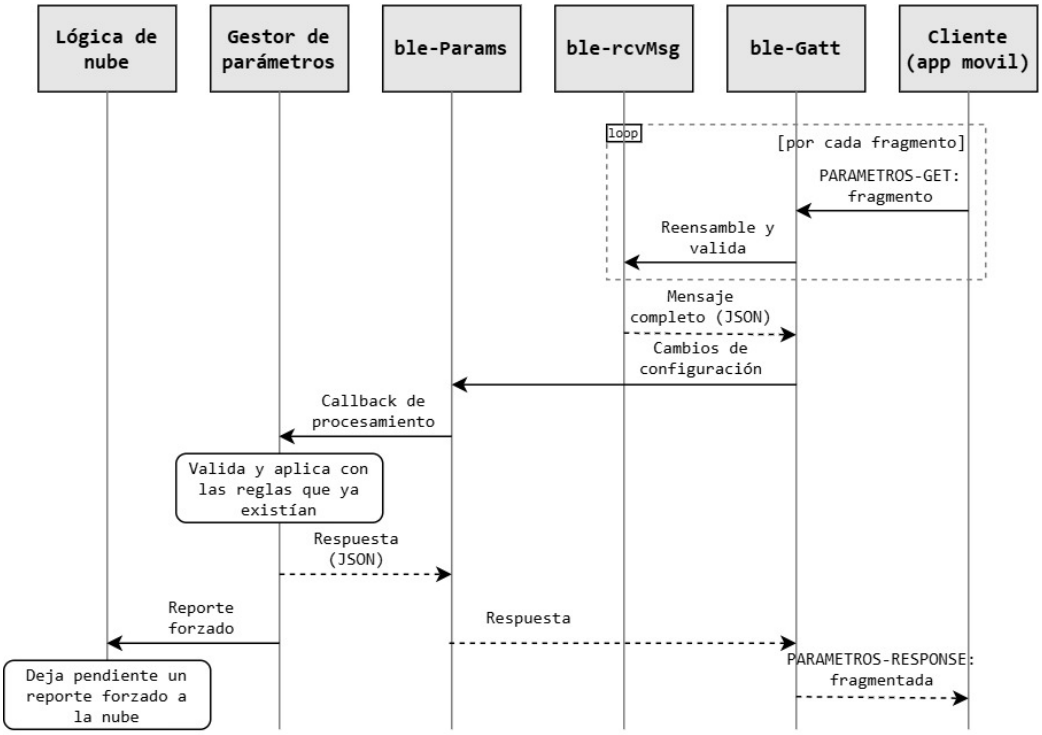


FIGURA 3.5. Escritura de parámetros desde la aplicación móvil. Los fragmentos recibidos se reensamblan, el gestor de parámetros valida y aplica los cambios con sus reglas existentes, la respuesta vuelve a la aplicación y queda pendiente un reporte forzado a la plataforma.

3.7.3. Pruebas de actuadores

Las pruebas de actuadores permiten que el técnico verifique en el lugar el funcionamiento de la heladera. Los comandos recibidos por `COMMAND-GET` se entregan mediante un callback a las rutinas de control existentes del N150, que habilitan el modo de prueba y accionan de forma individual el compresor, la resistencia, el ventilador y la luz, o ejecutan la rutina de prueba completa. El argumento del comando indica si la prueba se inicia o se detiene. El resultado que devuelve la rutina se informa a la aplicación por `COMMANDS-RESPONSE`. De este modo, la interfaz local no accede directamente a los actuadores, sino que utiliza las rutinas de prueba que el controlador ya tenía implementadas.

La figura 3.6 muestra el recorrido genérico de un comando. Las lecturas de metadatos y diagnóstico, descritas a continuación, siguen el mismo recorrido, con la diferencia de que la solicitud llega por `COMMAND-DATA` y la respuesta se transmite fragmentada por indicaciones.

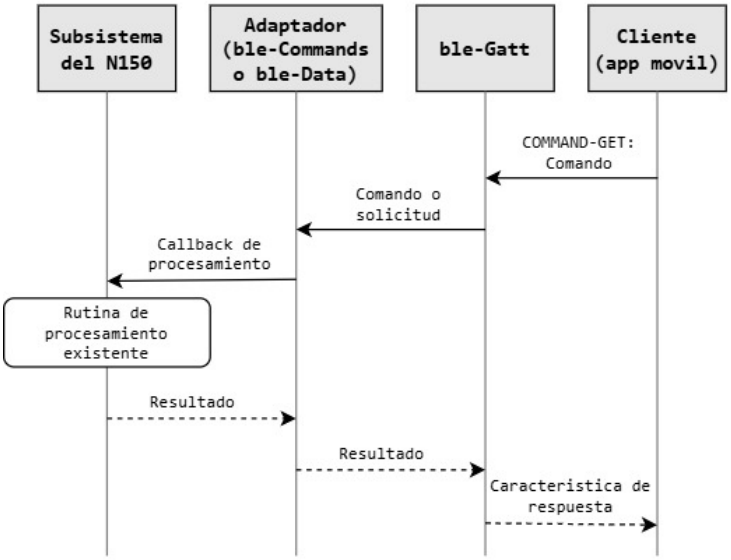


FIGURA 3.6. Recorrido genérico de un comando. La solicitud se delega mediante un callback a la rutina existente del subsistema correspondiente, y el resultado vuelve a la aplicación por la característica de respuesta asociada.

3.7.4. Metadatos y diagnóstico

Los metadatos y el diagnóstico se obtienen mediante callbacks que devuelven un documento JSON. Para los metadatos, el firmware reutiliza la información que la lógica de nube y conectividad remota ya construía para la plataforma, como el modelo y el serial del controlador, las versiones de firmware y los datos del módem y de la red celular. El diagnóstico reúne el estado de la conectividad del equipo, como la conexión activa, el registro en la red celular y la conexión con la plataforma, para que el técnico pueda identificar en el lugar por qué un equipo no reporta. En ambos casos, `ble-Data` copia el documento, `ble-Gatt` lo transmite fragmentado por indicaciones y el resultado de la operación se informa por `READ-STATUS`.

3.7.5. Comandos para la lógica de nube y conectividad remota

Algunas operaciones, como la desactivación de la conectividad celular, ya existían como comandos que el N150 recibe desde la plataforma. Para no duplicarlas, la interfaz BLE acepta esos mismos comandos por `CLOUDCOMMANDS-GET`. El mensaje, en formato JSON, se reensambla y se entrega mediante un callback a la cola de comandos de la lógica de nube y conectividad remota. A partir de ese punto, el comando sigue el mismo camino de validación y ejecución que uno recibido de forma remota.

3.7.6. Registros

La interfaz ofrece tres canales de registros, que se activan cuando la aplicación se suscribe a la característica correspondiente. `LOG-SEND` toma las entradas de la cola del sistema de registros del N150 y las transmite con el formato JSON de ese sistema. `LOG-TERMINAL` intercepta la salida de terminal del firmware mientras la aplicación está suscrita y la almacena en un búfer circular de 64 entradas de hasta 256 bytes, ubicado en la PSRAM cuando está disponible; si el búfer se llena, se descartan las entradas más antiguas. Por último, `LOG-CONTROL` es un canal independiente que los módulos del firmware pueden utilizar para informar eventos y el estado de un proceso en curso, sin consumir la cola general de registros.

Estos canales permiten observar el comportamiento del N150 durante una intervención de soporte sin depender de la conectividad celular ni de una conexión física a la terminal del equipo.

Capítulo 4

Ensayos y resultados

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se puedo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

[1] Sensify Solutions SA. *Sensify*. 2026. URL: https://sensifycloud.com/ (visitado 20-04-2026).

[2] *N150 Technical Reference Manual*. Sensify S. A. S. 2024. URL: https://www.sensify.com.ar/manuales/N150_general.pdf (visitado 04-09-2026).

[3] Swift Sensors. *Refrigeration Monitoring*. 2026. URL: https://www.swiftsensors.com/application/refrigeration-monitoring/ (visitado 20-04-2026).

[4] Monnit. *Remote Monitoring Solutions with Wireless Sensors for IoT*. 2026. URL: https://www.monnit.com/ (visitado 20-04-2026).

[5] Coel. *X35P Controlador para Refrigeração*. 2026. URL: https://www.coel.com.br/produto/x35p-controlador-para-refrigeracao/ (visitado 20-04-2026).

[6] AoFrio. *Soluciones IoT para refrigeración comercial*. 2026. URL: https://www.aofrio.com/es/ (visitado 04-09-2026).

[7] AoFrio. *AoFrio iQ para equipos de bebidas frías*. 2026. URL: https://www.aofrio.com/es/solutions/aofrio-iq-for-cold-drinks-equipment (visitado 04-09-2026).

[8] ASHRAE. *ASHRAE Handbook: Refrigeration*. ASHRAE, 2022. URL: https://www.ashrae.org/technical-resources/ashrae-handbook (visitado 10-09-2026).

[9] *PIC24FJ64GU205 Family Data Sheet*. DS40002204D. Microchip Technology Inc. 2021. URL: https://ww1.microchip.com/downloads/en/DeviceDoc/PIC24FJ64GU205-Family-Data-Sheet-DS40002204D.pdf (visitado 04-09-2026).

[10] *ESP32 Technical Reference Manual*. Rev. 5.3. Espressif Systems. 2023. URL: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf (visitado 04-09-2026).

[11] *BG95 Cat M1/Cat NB2/EGPRS Series*. Quectel Wireless Solutions. URL: https://www.quectel.com/product/lpwa-bg95-cat-m1-cat-nb2-egprs-series/ (visitado 04-09-2026).

[12] *Bluetooth Core Specification Version 5.3*. Bluetooth SIG. 2021. URL: https://www.bluetooth.com/specifications/specs/core-specification-5-3/ (visitado 10-09-2026).

[13] *Bluetooth Low Energy – ESP-IDF Programming Guide*. Ver. 5.3.2. Espressif Systems. 2024. URL: https://docs.espressif.com/projects/esp-idf/en/v5.3.2/esp32/api-guides/ble/index.html (visitado 10-09-2026).